Functions

A function is a block of reusable code that performs a specific task it is defined once but it can be called or used multiple times in a program

why we use functions

main reason reusability once we define the function we can call multiple times

organised and readability- functions break big program into smaller parts making them easier to read and understand

Avoid Repetation- if we need the same logic at multiple places you dont need to copy and paste

Types of function

Build-in functon

Already provided by python you can use them directly without defining

eg:-print(),sqrt(), max(), min(), len() etc

user defined function

functions that you create using def

In [1]:
```
def hello():
    print("Good evening")
hello()
```

```
Good evening
```

functions are case sensitive

if we define hello with lower case letterr but we are calling upper case H hello it returns error

In [3]:
```
def hello():
    print("Good evening")
Hello()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-3-b14830f9b4d3> in <module>
      1 def hello():
      2     print("Good evening")
----> 3 Hello()

NameError: name 'Hello' is not defined
```

In [6]:
```
def hello():
    print("Good evening")
hello()
def hello():
    print("Good evening")
hello()
def hello():
    print("Good evening")
hello()
```

```
Good evening
Good evening
Good evening
```

if we want the same logic multiple times define it once and call it many times

In [8]:
```
def hello():
    print("Good evening")
hello()
hello()
hello()
```

```
Good evening
Good evening
Good evening
```

passing with argument

when you pass arguments to a function you give it some inputs values so that the function can use them inside the code

In [9]:
```
def add(x,y):
    c=x+y
    print(c)
add(5,6)
```

```
11
```

In [8]:
```
def add(x,y,z):
    c=x+y+z
    print(c)
add(5,6)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-8-9f181b26e514> in <module>
      2     c=x+y+z
      3     print(c)
----> 4 add(5,6)

TypeError: add() missing 1 required positional argument: 'z'
```

we pass three arguments but we only call 2 arguments

In [22]:
```
def add(x,y,z):
    c=x+y+z
    print(c)
add(5,6,7)
```

```
18
```

while using functions instead of print use return

Use print() only to show output to the user.

Use return when you want to use the value again in the program.

In [17]:
```
def add(x,y):
    c=x+y
    return c
add(5,6)
```

Out[17]:
```
11
```

In [1]:
```
def add(a,b):
    print(a+b)
result=add(5,10)
print("Result is:",result)
```

```
15
Result is: None
```

In [2]:
```
def add(a,b):
    return a+b
result=add(5,10)
print("Result is:",result)
```

```
Result is: 15
```

In [18]:
```
def hello():
    print("Good evening")
hello()
def add(x,y):
    c=x+y
    return c
add(5,6)
```

```
Good evening
```

Out[18]: 11

In [19]:
```
def hello():
    print("Good evening")
def add(x,y):
    c=x+y
    return c
hello()
add(5,6)
```

```
Good evening
```

Out[19]: 11

In [20]:
```
# clean code
```

In [23]:
```
def add_sub(x,y):
    c=x+y
    d=x-y
    return c,d
result=add_sub(4,5)
result1=add_sub(4,5)
print(result)
print(result1)
```

```
(9, -1)
(9, -1)
```

by default we are getting output in tuple to avoid that store it in two variables insted of one variable

In [25]:
```
print(type(result))
print(type(result1))
```

```
<class 'tuple'>
<class 'tuple'>
```

In [4]:
```
def add_sub(x,y):
    c=x+y
    d=x-y
    return(c,d)
add, sub=add_sub(4,5)
print("addition:",add)
print("subtraction:",sub)
```

```
addition: 9
subtraction: -1
```

In [6]:
```
def sum(x,y):
    c=x+y
    d=x-y
    return(c,d)
add,sub=sum(4,5)
print("addition:",add)
print("subtraction:",sub)
```

```
addition: 9
subtraction: -1
```

if user define the function with 2 arguments must call 2 arguments

function argument is divident into 2 parts

formal argument and actual argument

In [28]:
```
## functions are reusable one we define the funtion we can call anytime
```

In [29]:
```
def update():
    x=8
    print(x)
update()
```

```
8
```

In [30]:
```
def add(a,b):  # a,b are called as formal argument
    c=a+b
    print(c)
add(5,6)  # 5&6 are called as actual argument
```

```
11
```

formal argument-formal argument is a variable you write inside the functon definition it acts as a placeholder for the value you give when you call the function

an actual argument is the real value you pass to a function when you call it

actual argument

postional argument

keyword

default

variable length

we mention at the time aof defination-formal

while we are calling-actual

Positional Argument

A positional argument means:

The value you pass to a function is matched to the parameter by its position (order).

The first value goes to the first parameter, the second value goes to the second parameter, and so on.Positional Argument

In [34]:
```
def person(name,age):
    print(name)
    print(age)
print(22,"nit")
```

```
22 nit
```

In [38]:
```
def person(name,age):
    print(name)
    print(age-1)
person(22,"nit")
```

```
22

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-38-f807a1d0eda9> in <module>
      2     print(name)
      3     print(age-1)
----> 4 person(22,"nit")

<ipython-input-38-f807a1d0eda9> in person(name, age)
      1 def person(name,age):
      2     print(name)
----> 3     print(age-1)
      4 person(22,"nit")

TypeError: unsupported operand type(s) for -: 'str' and 'int'
```

In [39]:
```
def person(name,age):
    print(name)
    print(age-1)
person("nit")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-39-8be0b0b9d2b9> in <module>
      2     print(name)
      3     print(age-1)
----> 4 person("nit")

TypeError: person() missing 1 required positional argument: 'age'
```

In [40]:
```
def person(name,age):
    print(name)
    print(age-1)
person(22)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-40-f1c2e1b76460> in <module>
      2     print(name)
      3     print(age-1)
----> 4 person(22)

TypeError: person() missing 1 required positional argument: 'age'
```

In [41]:
```
def person(name):
    print(name)
    print(age-1)
person(22,"nit")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-41-faffe9a98d8d> in <module>
      2     print(name)
      3     print(age-1)
----> 4 person(22,"nit")

TypeError: person() takes 1 positional argument but 2 were given
```

keyword argument

you specify the parameter name when calling the function order doesnt matter python matches the values using the parameter names

In [43]:
```
def person(name,age):
    print(name)
    print(age)
print(22,"nit")
```

```
22 nit
```

In [45]:
```
def person(name,age):
    print(name)
    print(age)
person(age=22,name="nit")
```

```
nit
22
```

In [46]:
```
def person(name,age):
    print(name)
    print(age+1)
person(age=22,name="nit")
```

```
nit
23
```

In [47]:
```
def person(name,age):
    print(name)
    print(age)
person(age=22,name="nit",phone=63330)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-47-137a9a42932b> in <module>
      2     print(name)
      3     print(age)
----> 4 person(age=22,name="nit",phone=63330)

TypeError: person() got an unexpected keyword argument 'phone'
```

In [49]:
```
def person(name,age,phone):
    print(name)
    print(age)
    print(phone)
person(age=22,name="nit",phone=63330)
```

```
nit
22
63330
```

default argument

a default argument is a parameter that already assgined value in afunction definition if we dont pass a value while calling the function python will use the default value if we pass a value while calling the funtion it will ovweride the default

In [53]:
```
def person(name,age):
    print(name)
    print(age)
person("nit")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-53-a2b54f9af0b4> in <module>
      2     print(name)
      3     print(age)
----> 4 person("nit")

TypeError: person() missing 1 required positional argument: 'age'
```

In [54]:
```
def person(name,age=19):
    print(name)
    print(age)
person("nit")
```

```
nit
19
```

In [55]:
```
def person(name,age=19):
    print(name)
    print(age)
person("nit",40)
```

```
nit
40
```

In [ ]:
```

```